



FOR SPRING DEVELOPERS

Rejected. Still Saved.

Spring Boot to Go

Spring Boot owns the lifecycle decisions. In Go, your application does.

The same payments API built twice, the same tables and the same SQL. Everything on this sheet was built and measured, not assumed.

Spring Boot 4.1.1 on Java 25; Go 1.27.1 with net/http, database/sql and the pgx driver; Postgres 17.

Every version on this sheet was current on 27 September 2026 and will not stay current. The method is the durable part: run the script, read what came back, and write down the result rather than the expectation.

In scope

- Control flow, routing and middleware
- Wiring by hand, the typed nil, interfaces and testing
- Errors, transactions and the request context
- Goroutine lifetime and shutdown

Out of scope

- Generics, and Go web frameworks
- Performance benchmarks, which were not measured

Footprint numbers are one machine, not a benchmark, and not a reason to choose either.

01 The findings, in one table

Error written, no return	Go: 400 and the row written
Two variable names on one path	Go panics at registration; Spring 500s
Service built out of order	nil check passed; first request panicked
No deferred rollback, request context	rescued: the pool answered
The same, on context.Background	pool exhausted, 5 s wait
Client gives up after 1 s	Spring still running; Go cancelled
Receipt on the request context	0 written, context canceled
SIGTERM mid-request, plain ListenAndServe	connection dropped

Every row is reproduced by `scripts/verify.sh`.

02 Writing an error is not returning

A payment of minus five got HTTP 400, amount must be positive, and the payment was written anyway. The handler called `http.Error` and carried on. Go's documentation for `http.Error`: it does not otherwise end the request. In our Spring handler, throwing the exception unwound the call before the service ran.

The same request, without the return

```
$ curl -s -w '\nHTTP %{http_code}\n' -X POST \
  localhost:18199/payments -H 'Content-Type:
  application/json' -d '{"amount":-5,"currency":"USD"}'
amount must be positive
{"id":2,"amount":-5,"currency":"USD","status":"CAPTURED"}
HTTP 400
$ psql -tAc "select id, amount from payments where amount < 0"
2|-5
```

handlers.go: the handler, with the return

```
func (h *Handlers) create(w http.ResponseWriter, r *http.Request) {
    var in NewPayment
    dec := json.NewDecoder(r.Body)
    dec.DisallowUnknownFields()
    if err := dec.Decode(&in); err != nil {
        http.Error(w, err.Error(), http.StatusBadRequest)
        return
    }
    if in.Amount <= 0 {
        http.Error(w, "amount must be positive", http.StatusBadRequest)
        return
    }
    p, err := h.payments.Create(r.Context(), in)
    if errors.Is(err, ErrOverLimit) {
        http.Error(w, err.Error(), http.StatusUnprocessableEntity)
        return
    }
    if err != nil {
        http.Error(w, err.Error(), http.StatusInternalServerError)
        return
    }
    // The receipt outlives the request, so it must not inherit the request's
    // cancellation. WithoutCancel keeps its values and drops its lifetime.
    h.receipts.SendAsync(context.WithoutCancel(r.Context()), p)
    writeJSON(w, http.StatusCreated, p)
}
```

It does not otherwise end the request; the caller should ensure no further writes are done to `w`. (`http.Error`)

Go 1.27.1 standard library documentation, checked 2026-09-27



Go: 400 and the row written. Spring: 400 and nothing written.

Run on a real machine, 2026-09-27. Reproduce it with scripts/verify.sh in the course repository.

03 Routing, familiar until it is not

Since Go 1.22 the standard router matches methods and path variables, and the most specific pattern wins in any registration order, as in Spring. Two variable names on one path overlap with neither more specific: Go panics while registering the routes; our Spring app started and failed the first matching request with a 500.

main.go: routes

```
mux := http.NewServeMux()
mux.HandleFunc("GET /payments/{id}", handlers.get)
mux.HandleFunc("GET /payments/latest", handlers.latest)
mux.HandleFunc("POST /payments", handlers.create)
mux.HandleFunc("GET /reports/slow", handlers.slow)
```

Adding GET /payments/{ref}

```
# the Go log, at startup
panic: pattern "GET /payments/{ref}" (registered at
/src/main.go:40) conflicts with pattern "GET /payments/{id}"
(registered at /src/main.go:38):
```

If two or more patterns match a request, then the most specific pattern takes precedence. (ServeMux)

Go 1.27.1 standard library documentation, checked 2026-09-27

Spring: IllegalStateException, Ambiguous handler methods mapped for '/payments/7', HTTP 500.

Run on a real machine, 2026-09-27. Reproduce it with scripts/verify.sh in the course repository.

04 No container: construction is your code

Main builds everything bottom up, and each constructor asks for what it needs, so the compiler makes main pass something. Built out of order, a nil service passed a nil check and the first request panicked: an interface holding a nil pointer is not a nil interface. Our Spring twin with the bean missing refused to start.

```
main.go: wired by hand, bottom up
```

```
// Bottom up: each constructor asks for what it needs.  
store := NewStore(db)  
receipts := NewReceipts(db)  
payments := NewPaymentService(store)  
handlers := NewHandlers(payments, receipts)
```

```
The typed nil
```

```
var service *PaymentService  
var payments Payments = service  
fmt.Println(payments == nil) // false: the interface holds a type
```

The nil check did not fire; the request panicked with a nil pointer dereference.

Run on a real machine, 2026-09-27. Reproduce it with `scripts/verify.sh` in the course repository.



05 Interfaces pay you back

The handlers depend on a small interface declared where it is used, satisfied implicitly by the service and by a fake. A test drives the real handler with httptest, no database and no server. It fails on the copy without the return: Create was called 1 time(s) for a rejected payment.

handlers.go: the interface, declared where it is used

```
type Payments interface {
    Create(ctx context.Context, in NewPayment) (Payment, error)
    Get(ctx context.Context, id int64) (Payment, error)
    Latest(ctx context.Context) (Payment, error)
    Report(ctx context.Context, seconds int) error
}

func NewHandlers(payments Payments, receipts ReceiptSender) *Handlers {
    if payments == nil || receipts == nil {
        panic("NewHandlers: a dependency is nil")
    }
}
```

handlers_test.go

```
func TestNegativeAmountIsRejectedAndNotCreated(t *testing.T) {
    fake := &fakePayments{}
    h := NewHandlers(fake, noReceipts{})
    req := httptest.NewRequest(http.MethodPost, "/payments",
        strings.NewReader(`{"amount":-5,"currency":"USD"}`))
    rec := httptest.NewRecorder()

    h.create(rec, req)

    if rec.Code != http.StatusBadRequest {
        t.Fatalf("status = %d, want 400", rec.Code)
    }
    if len(fake.created) != 0 {
        t.Fatalf("Create was called %d time(s) for a rejected payment",
            len(fake.created))
    }
}
```

Passed on the app, failed on the copy without the return.

Run on a real machine, 2026-09-27. Reproduce it with scripts/verify.sh in the course repository.

06 Transactions are yours

Begin on the request context, defer the rollback, commit. Without the deferred rollback, four failures in a pool of four were rescued by the request context, because database/sql rolls a transaction back when its context is cancelled. Begun on context.Background, the pool was exhausted and the next read waited five seconds. Spring's interceptor rolls back on runtime exceptions and errors by default, not checked exceptions.

```
service.go: the transaction is yours

func (s *PaymentService) Create(ctx context.Context, in NewPayment) (Payment, error) {
    tx, err := s.store.db.BeginTx(ctx, nil)
    if err != nil {
        return Payment{}, fmt.Errorf("begin: %w", err)
    }
    defer tx.Rollback() // harmless once Commit has succeeded

    p := Payment{Amount: in.Amount, Currency: in.Currency, Status: "CAPTURED"}
    err = tx.QueryRowContext(ctx,
        `INSERT INTO payments (amount, currency, status) VALUES ($1, $2, $3)
        RETURNING id`, p.Amount, p.Currency, p.Status).Scan(&p.ID)
    if err != nil {
        return Payment{}, fmt.Errorf("insert payment: %w", err)
    }
    if p.Amount > limit {
        return Payment{}, ErrOverLimit
    }
    _, err = tx.ExecContext(ctx,
        `INSERT INTO audit (payment_id, note) VALUES ($1, 'captured')`, p.ID)
    if err != nil {
        return Payment{}, fmt.Errorf("audit payment %d: %w", p.ID, err)
    }
    return p, tx.Commit()
}
```

If the context is canceled, the sql package will roll back the transaction. (BeginTx)

Go 1.27.1 standard library documentation, checked 2026-09-27

07 Whose lifetime is this work?

A client gave up one second into a six second query. One second later the query was still running in our Spring app, cancelled in Go, and running again in Go on a background context. The request context carries the client's lifetime down to the database with pgx, in this app. And the same tool points the other way: a receipt goroutine given the request context wrote nothing, context canceled; detached with its own ten second bound and tracked in a wait group, it wrote.

handlers.go: the request context goes to the database

```
func (h *Handlers) slow(w http.ResponseWriter, r *http.Request) {
    seconds, _ := strconv.Atoi(r.URL.Query().Get("seconds"))
    start := time.Now()
    if err := h.payments.Report(r.Context(), seconds); err != nil {
        http.Error(w, err.Error(), http.StatusServiceUnavailable)
        return
    }
    writeJSON(w, http.StatusOK, map[string]any{
        "slept": time.Since(start).Round(time.Second).String()})
}
```

The client gives up after one second: what is still running?

```
Spring:    1
Go:        0
Background: 1
```

receipts.go: detached work, bounded and tracked

```
func (r *Receipts) SendAsync(ctx context.Context, p Payment) {
    r.inFlight.Go(func() {
        ctx, cancel := context.WithTimeout(ctx, 10*time.Second)
        defer cancel()
        if err := r.send(ctx, p); err != nil {
            log.Printf("receipt for payment %d: %v", p.ID, err)
        }
    })
}

func (r *Receipts) Wait(ctx context.Context) error {
    done := make(chan struct{})
    go func() {
        r.inFlight.Wait()
        close(done)
    }()
    select {
    case <-done:
        return nil
    case <-ctx.Done():
        return ctx.Err()
    }
}
```

For incoming server requests, the context is canceled when the client's connection closes, the request is canceled, or when the ServeHTTP method returns.

Go 1.27.1 standard library documentation, checked 2026-09-27



08 Shutdown is yours

SIGTERM one second into a three second request: plain ListenAndServe dropped the connection; a server that catches the signal and calls Shutdown finished it; Spring Boot finished it, graceful by default. Shutdown waits for HTTP connections, not for goroutines the application started, so the receipts are waited for inside the same bound.

main.go: shutdown, in order

```
func serve(srv *http.Server, receipts *Receipts) {
    ctx, stop := signal.NotifyContext(context.Background(),
        os.Interrupt, syscall.SIGTERM)
    defer stop()
    go func() {
        err := srv.ListenAndServe()
        if err != nil && !errors.Is(err, http.ErrServerClosed) {
            log.Fatal(err)
        }
    }()
    log.Printf("listening on %s", srv.Addr)
    ←ctx.Done()
    log.Print("shutting down, waiting for in-flight requests")
    shutdown, cancel := context.WithTimeout(context.Background(), 30*time.Second)
    defer cancel()
    if err := srv.Shutdown(shutdown); err != nil {
        log.Print(err)
    }
    // Server.Shutdown knows about HTTP connections, not about goroutines this
    // application started. Those are ours to wait for, inside the same bound.
    if err := receipts.Wait(shutdown); err != nil {
        log.Printf("receipts still in flight: %v", err)
    }
    log.Print("stopped")
}
```

SIGTERM one second into a three second request

```
SIGTERM one second into a three second request:
go, ListenAndServe only: HTTP 000 curl exit 52
go, with Shutdown:      {"slept":"3s"}
HTTP 200
spring boot default:    {"slept":"3s"} HTTP 200
```

Graceful shutdown is enabled by default with all three embedded web servers.

Spring Boot reference documentation, checked 2026-09-27

09 The interview questions this answers

Why did a 400 still save the payment?

What is inside an interface value, and when is it nil?

Why is the rollback deferred, and when is it harmless?

What does the request context do when the client disconnects?

When should work not inherit the request context?

What does Server.Shutdown wait for, and what does it not?

For this transaction, query, goroutine and process: where does the lifetime come from?

Before a Go service goes near production

1. Every error response is followed by a return
2. Dependencies are built in order, in one place, where a reader can see them
3. Every transaction defers its rollback
4. The request context is passed down to every query that belongs to the request
5. Work that outlives the request is detached, bounded and tracked
6. The server is constructed with timeouts, and shuts down on SIGTERM within a bound
7. scripts/verify.sh passes on a fresh clone

Everything else

- | | |
|--|---|
|  Site
https://code-with-sam-dev.github.io |  GitHub
https://github.com/code-with-sam-dev |
|  YouTube
https://www.youtube.com/@CodewithSam-Dev |  Buy me a coffee
https://buymeacoffee.com/codewithsam |
|  LinkedIn
https://www.linkedin.com/company/code-with-sam-dev |  Buy me sadza
https://buymesadza.com/codewithsam |
|  x
https://x.com/CodeWithSamDev | The course repository
https://github.com/code-with-sam-dev/spring-to-go |
|  TikTok
https://www.tiktok.com/@codewithsamdev | net/http package
https://pkg.go.dev/net/http |
|  Instagram
https://www.instagram.com/codewithsamdev | database/sql package
https://pkg.go.dev/database/sql |
|  Facebook
https://www.facebook.com/codewithsamdev | Spring Boot graceful shutdown
https://docs.spring.io/spring-boot/reference/web/graceful-shutdown.html |

If this was useful, a like and a subscribe help more than you would think. And tell me in the comments which part of your workflow you want taken apart next.

Java is a trademark of Oracle. Spring and Spring Boot are trademarks of Broadcom. Go is a trademark of Google LLC. PostgreSQL is a trademark of the PostgreSQL Community Association. This is an independent, unofficial guide produced by Code with Sam, not affiliated with or endorsed by any of them, and no third party artwork is reproduced here.

Technical claims verified 2026-09-27 against the documentation named beside each one. Vendors change behaviour and defaults between versions, so check the version you are actually running before you rely on any of it.